CERTIFIED SUB-THRESHOLD LOGICAL ERROR RATES: EXACT UNCORRECTABLE-SET COUNTS AND A TWO-SIDED RATIONAL BRACKET FOR THE ROTATED SURFACE CODE, REPLAYABLE IN THE STANDARD LIBRARY

DANIEL KIRTCHAKOV

Date: August 11, 2026.

2020 *Mathematics Subject Classification.* Primary 81P73; Secondary 94B05, 68V15, 60C05.

Key words and phrases. Quantum error correction, surface code, circuit-level noise, logical error rate, detector error model, lookup-table decoder, certified computation, Poisson binomial.

Computation and authorship. All detector-error-model construction, enumeration, verification, and drafting in this work were produced by **Claude** (Anthropic), directed by the author, on a single Apple laptop. The public artifact is a certificate together with a checker that imports only the Python standard library and rebuilds every certified quantity from the certificate alone. The checker is not code-independent of the private enumeration engine that produced the certificate. Of the 141 executable lines of `check_wedge.py`, 58 appear verbatim (up to indentation) in that engine, and lines 151 to 166, the body of the truncated-weight Poisson-binomial dynamic program that computes the tail T and hence the bracket width U minus L, are byte-identical to the engine's lines 124 to 139; that is the longest identical run in the file. The $d = 5$ checker stands in the same relation to its own engine, sharing 77 of its 143 executable lines, its tail program among them. The shared material is the re-derivation itself: the decoder breadth-first search, the joint-distance search, the subset-enumeration arithmetic, and the tail program. A passing check may therefore not be read, on its own, as an independent re-derivation of those quantities. This is a factual methods statement and it is part of the point of the note: the provenance of the *search* is designed to be irrelevant to the validity of the *bracket*.

That gap has since been closed for the tail at every certificate and for the whole bracket at $d = 3$, by two programs written against the definitions of this note rather than against either source. Neither reuses an implementation from the checkers or the engines; measured line by line, the tail program overlaps `check_wedge.py` in 8 of its 183 executable lines and the bracket program in 8 of its 141, and no overlap between either program and any of the four audited files exceeds twelve lines, every one of them an import, a loop or branch header, the main guard, or a single statement such as $U = L + T$. The tail is computed by a deliberately different route: the full probability generating function of the number of firing mechanisms, expanded to an exact integer polynomial over the product of the mechanism denominators, with T taken as the complement of the coefficients up to $W_{\max}$ and the coefficient sum checked against that product exactly. It reproduces T as a rational, numerator and denominator, at all five shipped certificates. At $d = 3$ the second program re-derives the whole bracket from the mechanism list alone: the decoder by its own breadth-first search, agreeing at all 256 syndromes; the uncorrectable counts 55, 690 and 4,078 at weights 2, 3 and 4; all 4,823 uncorrectable sets, set for set; and L, T and U exactly. At $d = 5$ the certificate carries per-weight SHA-256 digests of the uncorrectable sets rather than the sets themselves, and L at $d = 5$ has not been independently re-derived; only T is backstopped there.

The layer the tamper controls of the trust section rest on is original to the checkers: in both checkers, the hash recomputation, every comparison against the certificate, the containment test on the bracket, and the fail-and-exit reporting share no text with either engine. Two guards elsewhere in the $d = 5$ checker do match the $d = 5$ engine — `if not seen[s]:` and `if cnt != comb(m, w):`, where the engine raises and the checker fails — and no shared line of the $d = 3$ checker is a guard of that kind. The tamper controls are unaffected. The detector error models were generated with Stim 1.16.0 (Gidney), which is a generator and is *not* in the trusted base (§5).

Prior-art record. The primary source [MWB26] was read at its abstract and relevant sections on August 11, 2026; the passages quoted below are verbatim. Novelty was swept the same day against arXiv and the web over a 24-month window; the dated record ships with the artifacts. *Draft of August 11, 2026.*

Abstract. Can the logical error rate of a quantum error correcting code, in the deep sub-threshold regime that Mullan, Weippert, and Brown [MWB26] identify as inaccessible to direct Monte Carlo simulation, be bounded rigorously and *checkably* — by an artifact a skeptic replays without trusting the software that produced it? We give such an artifact for the distance-3 and distance-5 rotated surface codes under one round of circuit-level depolarizing noise and a fixed lookup-table (coset-leader) decoder. From the independent-mechanism detector error model we compute, in exact arithmetic, the integer counts A_w of uncorrectable weight-w fault sets up to a truncation weight $W_{\max}$, and convert them into a two-sided exact-rational bracket $L \le P_L \le U$ on the decoder's logical error probability, where the width $U - L$ is a Poisson-binomial tail computed exactly. A certificate carries the mechanism list and the counts; a standard-library Python checker re-derives the decoder, the counts, and both bounds from scratch and fails loudly on any mismatch. Deep sub-threshold, at $p = 10^{-3}$, the certified bracket is far tighter than a 10^7-shot Monte Carlo interval — by a factor of about 18,500 at $d = 3$ and about 626 at $d = 5$, where matching the bracket would take Monte Carlo on the order of 4×10^{12} shots. The advantage is a deep-sub-threshold phenomenon and it degrades as the expected number of firing mechanisms grows: at $p = 10^{-2}$ the bracket still beats Monte Carlo at $d = 3$ (by about 2.3×) but loses at $d = 5$ (by about 20×), because the weight-truncation tail is fat there — not because a threshold has been crossed, since the logical rate still falls with distance. The frontier is sharp: exact re-verification at weight 7 exceeds a pure-Python checker. We state precisely what is certified, what is merely trusted, and close with the questions the computation opens.

1. The question

A fault-tolerant quantum memory is judged by its *logical error rate* $P_L(p)$: the probability, per logical operation, that decoding fails, as a function of the physical error rate p. Below the code's threshold this rate is meant to fall steeply with the code distance d, and it is exactly this steep, small-P_L regime that matters for a working device — and exactly there that it is hardest to measure. Mullan, Weippert, and Brown [MWB26] put the difficulty plainly: at the error rates of interest, code "performance at these error rates is not amenable to direct Monte Carlo simulation" [MWB26, abstract], because the events being counted are too rare for a feasible number of shots to resolve. Their own response is an estimator — a pruning step followed by a family of Metropolis–Hastings "subregion MCMC" samplers, in the lineage of the Markov-chain method Bravyi and Vargo [BV14] adapted to quantum error correction. An estimator returns a number with a statistical error bar. It does not return a proof.

This note asks a different question. Not "what is P_L?" but: *can P_L be bracketed rigorously, by a rational interval a stranger can re-derive from a small certificate using nothing but a standard-library program?* The regime that defeats Monte Carlo — rare failures, deep sub-threshold — is the regime where exact counting is most favourable, because the objects to be counted, the low-weight fault configurations that the decoder mishandles, are few. We turn the rarity that blocks sampling into the finiteness that enables a certificate.

The construction is elementary and is stated in full in §2. Its output is, for each physical rate p, a pair of exact rationals $L \le U$ with

$$L \;\le\; P_L(p) \;\le\; U,$$

where $P_L(p)$ is the logical error probability of a *fixed* lookup-table decoder under the independent-mechanism fault model of the code's circuit-level detector error model (DEM). The certificate is the mechanism list and a handful of integers; the checker is a single file of standard-library Python. §4 reports the numbers and, honestly, both outcomes: where the bracket dominates Monte Carlo by four orders of magnitude, and where it loses. §5 separates what is machine-checked from what is trusted. §6 locates the wall — exact re-verification at weight seven — that the method hits and the

next engine must break. We claim no new value of any P_L and no threshold; we claim a replayable object and a quantitative account of when it is worth having.

2. The certified object

2.1. **Model.** Fix a detector error model with n detectors, a single logical observable, and m independent fault *mechanisms*. Mechanism $j \in [m]$ carries a detector mask $d_j \in \mathbb{F}_2^n$, an observable bit $o_j \in \mathbb{F}_2$, and a probability $p_j \in (0,1)$; the p_j are the *exact* dyadic rationals of the IEEE-754 doubles emitted by Stim's `detector_error_model`, so every denominator is a power of two and all arithmetic below is exact. A *fault configuration* is a subset $F \subseteq [m]$ of mechanisms that fire. Under the independent-mechanism model the m mechanisms fire independently, so

$$\mathbb{P}(F) \;=\; \prod_{j\in F} p_j \prod_{j\notin F}(1-p_j), \tag{1}$$

and F produces syndrome $\sigma(F) = \bigoplus_{j\in F} d_j \in \mathbb{F}_2^n$ and true observable flip $\mathrm{obs}(F) = \bigoplus_{j\in F} o_j \in \mathbb{F}_2$. The *weight* of F is $|F|$.

2.2. **Decoder.** The decoder is the minimum-cardinality lookup table (the coset-leader decoder of Tomita and Svore [TS14]). Run a breadth-first search over the syndrome space $\mathbb{F}_2^n$ from the zero syndrome, expanding by the m mechanism masks in a fixed canonical index order, first assignment winning. This assigns to every reachable syndrome s a minimum-cardinality fault configuration reaching s; let $D(s) \in \mathbb{F}_2$ be that configuration's observable flip. Then $D : \mathbb{F}_2^n \to \mathbb{F}_2$ is a fixed function, and

$$F \text{ is } \textit{correctable} \iff D(\sigma(F)) = \mathrm{obs}(F).$$

We emphasise what this decoder is not: it ranks fault configurations by their *cardinality* $|F|$, treating all m mechanisms as equally likely, and so it is neither the maximum-likelihood decoder nor minimum-weight perfect matching. Every bound below is a statement about *this* lookup-table decoder; none of it bounds the code's optimal-decoder logical rate.

2.3. **The logical error probability and its counts.** The decoder fails on F exactly when F is uncorrectable, and distinct configurations F are disjoint events, so

$$P_L \;=\; \sum_{\substack{F\subseteq[m]\\ F \text{ uncorrectable}}} \mathbb{P}(F), \qquad A_w \;:=\; \#\{\, F : |F| = w,\ F \text{ uncorrectable}\,\}. \tag{2}$$

The counts A_w depend only on the masks (d_j, o_j) and the decoder D, not on the probabilities p_j: they are a fixed, p-independent integer invariant of the DEM-plus-decoder pair.

2.4. **The two-sided bracket.** Choose a truncation weight $W_{\max}$. Define the exact rational

$$L \;=\; \sum_{\substack{F \text{ uncorrectable}\\ |F|\le W_{\max}}} \mathbb{P}(F). \tag{3}$$

Let $X_1, \dots, X_m$ be independent with $X_j \sim \mathrm{Bernoulli}(p_j)$, let $W = \sum_j X_j$ be the total number of firing mechanisms, and set

$$T \;=\; \mathbb{P}(W \ge W_{\max} + 1). \tag{4}$$

Theorem 2.1 (Two-sided bracket)**.** *$L \le P_L \le L + T$. Writing $U = L + T$, the interval $[L, U]$ is exact and its width is $U - L = T$.*

Proof. By (2) and (3), $P_L - L = \sum_{F \text{ unc}, |F|>W_{\max}} \mathbb{P}(F) \geq 0$, which is the lower bound. For the upper bound, every uncorrectable F with $|F| > W_{\max}$ has $|F| \geq W_{\max}+1$, and the event "exactly F fires" entails $W = |F| \geq W_{\max}+1$; these events are disjoint across F, so

$$P_L - L = \sum_{\substack{F \text{ unc} \\ |F|>W_{\max}}} \mathbb{P}(F) \;\leq\; \sum_{\substack{F\subseteq[m] \\ |F|\geq W_{\max}+1}} \mathbb{P}(F) \;=\; \mathbb{P}(W \geq W_{\max}+1) = T. \qquad \square$$

The upper bound is deliberately a union-type bound: it charges the entire high-weight tail, correctable or not, against $P_L - L$. The tail T is a Poisson-binomial upper tail and is computed exactly by a dynamic program in $O(m\, W_{\max})$ rational operations, carrying the states $\mathbb{P}(W = k)$ for $k \leq W_{\max}$ and $\mathbb{P}(W \geq W_{\max}+1)$. The lower bound L is accumulated in exact integer arithmetic over the common denominator $D = \prod_j p_{\text{den},j}$; writing $m_j = p_{\text{den},j} - p_{\text{num},j}$ and $M = \prod_j m_j$, each term is $\mathbb{P}(F) = \big(\prod_{j\in F} p_{\text{num},j}\big)\,(M/\prod_{j\in F} m_j)/D$, where $\prod_{j\in F} m_j$ divides M so the division is exact — avoiding a greatest-common-divisor reduction on each of millions of terms with thousand-bit denominators.

2.5. **Why the bracket is sharp exactly sub-threshold.** Let $w^\star = \min\{w : A_w > 0\}$ be the least uncorrectable weight. As $p \to 0$ with the DEM's rates scaling proportionally, (2) gives $P_L = \Theta(p^{w^\star})$ (leading term $A_{w^\star} p^{w^\star}$), while $T = \Theta(p^{W_{\max}+1})$ (leading term $\binom{m}{W_{\max}+1} p^{W_{\max}+1}$). Hence the *relative* width satisfies

$$\frac{U-L}{P_L} \;=\; \frac{T}{P_L} \;=\; \Theta\big(p^{W_{\max}+1-w^\star}\big), \qquad W_{\max}+1-w^\star > 0, \tag{5}$$

so the bracket tightens as a positive power of p as $p \to 0$. For the two codes below $W_{\max}+1-w^\star = 3$. A shot-based Monte Carlo estimate, by contrast, resolves $P_L = \Theta(p^{w^\star})$ to fixed relative precision only with a shot count growing like $1/P_L = \Theta(p^{-w^\star})$: the rarer the failure, the worse Monte Carlo does and the better the certificate does. This is the precise sense in which the method meets [MWB26] where they say Monte Carlo cannot go. The same formula (5) predicts where the certificate loses: the *absolute* width $T = \mathbb{P}(W \geq W_{\max}+1)$ is large whenever $\mathbb{E}[W] = \sum_j p_j$ is not small — for larger p, or for more mechanisms m — while a Monte Carlo confidence interval shrinks as $N^{-1/2}$ regardless. At fixed $W_{\max}$ this forces a crossover governed by $\mathbb{E}[W]$ and m, not by the code threshold (§4).

3. Where this sits

Three neighbouring bodies of work compute logical error rates; none produces the object here. *Estimators*: the improved MCMC of [MWB26] and its Bravyi–Vargo antecedent [BV14] reach sub-threshold rates but return a sampled value with a statistical error bar, not a bracket a third party can re-derive. *Analytic approximations*: Regev, Dilley, Nutaro, Delgado, and Bennink [Reg26] give closed-form logical-error-rate approximations (their analysis includes measurement errors and a locally-correlated model), and Forlivesi, Valentini, and Chiani [FVC25] use MacWilliams weight enumerators, extending to noisy syndrome-extraction circuits; both are analytic (weight-enumerator or closed-form) bounds and asymptotics that loosen away from their validity window and carry no machine-checkable certificate, and their object — undetectable-error enumerators or a closed-form estimate — is not a decoder-specific count of uncorrectable configurations. *Formal verifiers*: Lean-QEC [LeanQEC26], via a verified SAT reduction, and Veri-QEC [VeriQEC25], via SMT, machine-check code *distance* and error-correction conditions — a Boolean property of the code — not a two-sided bracket on a *probability*. What is new here is the intersection: exact integer uncorrectable-set counts A_w, a two-sided exact-rational bracket on P_L under a stated decoder at circuit-level noise, and independent re-verification from a standard-library checker. We claim that intersection, and no value or threshold within it.

4. Results

We instantiate the construction on the rotated surface code (`rotated_memory_z`, one round, a single depolarizing parameter p on all four circuit-level noise channels) at $d = 3$ and $d = 5$. Table 1 records the two detector error models; the counts A_w are p-independent, and the circuit-level distance shown — the least weight $\leq W_{\max}$ of a fault set with zero syndrome and observable flip 1 (an undetectable logical error) — is re-derived by the checker.

code	detectors n	mechanisms m	dist.	uncorrectable counts A_w
$d = 3$ ($W_{\max} = 4$)	8	23	3	A_2=55, A_3=690, A_4=4078
$d = 5$ ($W_{\max} = 5$)	24	77	5	A_3=2728, A_4=154394, A_5=4057999

Table 1. The two certified detector error models. At $d = 5$ the 77 mechanisms are 12 of degree 1 and 65 of degree 2; $A_1 = A_2 = 0$, so the least uncorrectable weight is $w^\star = 3$. At $d = 3$, $A_1 = 0$ and $w^\star = 2$.

Table 2 gives the certified brackets at $p = 10^{-3}$ and $p = 10^{-2}$, alongside a pre-registered Monte Carlo kill test: 10^7 shots of the same DEM decoded with the same lookup table, and its 95% confidence interval. The rule, fixed before the runs: the bracket is LIVE at a point if it is narrower than the Monte Carlo interval there, DEAD if wider.

code	p	L	U	width T	vs. 10^7-shot MC
$d = 3$	10^{-3}	3.3589593231e−4	3.3589715988e−4	1.23e−9	≈ 18,500× narrower (LIVE)
$d = 3$	10^{-2}	2.7101553370e−2	2.7188139126e−2	8.66e−5	≈ 2.33× narrower (LIVE)
$d = 5$	10^{-3}	2.6135024167e−5	2.6144956889e−5	9.93e−9	≈ 626× narrower (LIVE)
$d = 5$	10^{-2}	1.6118428758e−2	1.9426205101e−2	3.31e−3	≈ 20.5× wider (DEAD)

Table 2. Certified brackets (decimal shadows of exact rationals) and the Monte Carlo kill test. The endpoints are exact; the checker prints the full numerator/denominator, here abbreviated. At $d = 5$, $p = 10^{-3}$, matching the bracket width by sampling would take Monte Carlo about 3.9×10^{12} shots; the bracket [2.6135e−5, 2.6145e−5] lies inside the 10^7-shot interval.

4.1. **Both verdicts, read honestly.** Deep sub-threshold, at $p = 10^{-3}$, the certificate dominates: about 18,500× tighter than the 10^7-shot interval at $d = 3$, and about 626× at $d = 5$, where Monte Carlo would need on the order of 4×10^{12} shots to match. This is the central message, and it is exactly the regime [MWB26] call inaccessible to sampling.

At the larger rate $p = 10^{-2}$ the picture splits, and the split is instructive. At $d = 3$ the bracket is still LIVE, about 2.33× narrower than Monte Carlo. At $d = 5$ it is DEAD, about 20.5× wider. The difference is not a threshold. The logical rate still *falls* with distance at $p = 10^{-2}$ — $P_L(d{=}5) \in [1.61\text{e}{-2}, 1.94\text{e}{-2}]$ lies below $P_L(d{=}3) \approx 2.72\text{e}{-2}$ — which is the below-threshold signature, not the above-threshold one. What changed is the truncation tail $T = \mathbb{P}(W \geq W_{\max} + 1)$: at $d = 5$ there are 77 mechanisms and $\mathbb{E}[W] = \sum_j p_j = 1.456$ at $p = 10^{-2}$, so more than one mechanism fires on average and the tail is fat; at $d = 3$ there are 23 mechanisms and $\mathbb{E}[W] = 0.481$, so the tail stays thin and the bracket keeps winning. By (5), the crossover where Monte Carlo overtakes is governed by $\mathbb{E}[W]$ and the mechanism count, and it is a large-m (here $d = 5$) phenomenon at the larger p — not a universal "above threshold" statement. We therefore describe the loose regime by its cause, the weight-truncation tail, and record the $d = 3$ contrast rather than generalising the $d = 5$ loss.

4.2. **A tighter optional run.** Pushing the truncation to $W_{\max} = 6$ at $d = 5,\ p = 10^{-3}$ adds the count $A_6 = 67{,}711{,}204$ and replaces the tail by $T = \mathbb{P}(W \geq 7) = 1.881 \times 10^{-10}$, tightening the bracket to [2.6138502142e−5, 2.6138690261e−5] (relative width about 7.2×10^{-6}). This certificate verifies, but its checker is slow (§6); it is offered as an optional sharpening, not part of the released $W_{\max} = 5$ core.

5. What is certified, and what is trusted

The value of a certificate is only as clear as the line between what a skeptic can replay and what they must take on faith. Here that line is drawn sharply.

Machine-checked, replayable with standard-library Python. The checker reads only the certificate. It (i) recomputes the SHA-256 of the canonically serialised mechanism list; (ii) rebuilds the coset-leader decoder D by breadth-first search over the syndrome space from syndrome 0, mechanisms applied in canonical index order, first assignment winning; (iii) enumerates all $\binom{m}{w}$ fault sets for $w \leq W_{\max}$, reproducing the counts A_w, the certificate's record of which fault sets are uncorrectable, and the exhaustion record; (iv) re-derives the circuit-level distance; and (v) recomputes L, T, and $U = L + T$ in exact arithmetic and compares them string-for-string. Any mismatch prints `CHECK FAIL` and exits nonzero.

One checker ships per distance, and steps (ii)–(iv) take different forms because the objects differ in size. At $d = 3$ the syndrome space has $2^8 = 256$ states and the uncorrectable sets of weight at most 4 number 4823 $(55 + 690 + 4078)$, so the certificate carries both in full: `check_wedge.py` runs the breadth-first search to exhaustion and compares the rebuilt table against the embedded decoder table entry by entry, compares the enumerated index tuples against the embedded uncorrectable-set lists, and obtains the distance from a separate joint search over (syndrome, observable) pairs. At $d = 5$ the table has 2^{24} entries and the uncorrectable sets number in the millions, so neither is embedded: `check_wedge_d5.py` truncates the search at depth $W_{\max}$ — sound because a weight-w fault set with $w \leq W_{\max}$ has a syndrome at breadth-first distance $\leq W_{\max}$, so its coset leader is the full-table leader — aborts if any enumerated syndrome is unreached, compares a per-weight SHA-256 over the uncorrectable index tuples, and reads the distance off the enumeration. The $d = 3$ certificate is thus the more explicit of the two artifacts, and its checker pins the decoder table on the whole syndrome space rather than on the weight-$\leq W_{\max}$ syndromes alone; the certified guarantee $L \leq P_L \leq U$ is the same in both cases.

Between them the checkers import only `hashlib`, `itertools`, `json`, `sys`, `fractions`, `math.comb`, and `collections`; they use no signals, subprocesses, network, or wall-clock, and run unchanged under a clean environment. An independent replay reproduced every certified number and rejected a battery of tamper controls (flipped counts, a perturbed probability, a corrupted decoder rebuild, altered bracket digits, a scope downgrade), each nonzero.

Trusted. Four things, stated plainly.

(1) *The mechanism list is the trust root.* Each checker certifies $L \leq P_L \leq U$ *given* the (d_j, o_j, p_j) list embedded in the certificate. It does not — and from the public artifact cannot — re-verify that this list *is* Stim's detector error model at the stated p; that binding lives in the private generator. This is the single link a stranger cannot re-check, and it is the reason the certificate ships the full mechanism list rather than a reference to it.

(2) *The independent-mechanism DEM is an approximation of the physical circuit.* The bracket bounds P_L of the DEM, which treats the depolarizing channels as independent single-mechanism events (Stim's standard DEM semantics). It is not, and does not claim to be, a bound on the true depolarizing circuit's P_L. Empirically the gap is small but real: at $d = 5,\ p = 10^{-3}$ a 10^7-shot Monte Carlo of the *raw circuit* returns a point estimate (2.81×10^{-5}) about 1.17 standard errors above U — statistically consistent, but a reminder

that "circuit-level" here names the DEM's origin, not a certified bound on the physical circuit.

(3) *CPython, the operating system, and the hardware.*
(4) *SHA-256 collision resistance*, for the bookkeeping only — a skeptic can regenerate and re-check every quantity without trusting any hash.

Stim, the enumeration engine, and the Monte Carlo are *not* in the trusted base: Stim's model is transcribed into the certificate and re-checked from there, the engine can be deleted and the checkers still verify, and the Monte Carlo is a falsification test, not an input to any bound.

6. The wall at weight seven, and certification

The method's reach is bounded by what a portable pure-Python checker can re-verify, and that boundary is close. At $d = 5$ the $W_{\max} = 5$ checker is comfortable, about 34 seconds. The $W_{\max} = 6$ checker is already heavy: on a stock laptop it exceeded a 600-second foreground budget (completing `CHECK PASS` in the background), so the honest figure is order ten-plus minutes, not "a few". The $W_{\max} = 7$ checker is the wall. There are $\binom{77}{7} = 2{,}404{,}808{,}340$ weight-7 fault sets to enumerate — about twenty-six minutes at the engine's rate — and the exact lower bound then accumulates tens of millions of big-integer terms with roughly 4600-bit denominators, pushing a pure-Python re-verification into hours. Breaking the wall is a computational-geometry-of-proofs problem, not a physics one: it needs either a proof-carrying weighted model counter (a CPOG-style certificate from a tool such as `d4+cpog`) that a small checker can validate, or a locality-exploiting exact convolution the checker can re-derive. Either would carry the same two-sided bracket to $W_{\max} = 7$ and beyond, and is the engine's next build target.

Certification. The permanent record of each result is a single certificate JSON — the mechanism list, the p-independent counts A_w, the uncorrectable-set record (explicit index tuples and the full decoder table at $d = 3$, per-weight hashes at $d = 5$), the exhaustion record, and the exact rational L, T, U — together with the standard-library checker that re-derives all of it. The two together are the public unit; they are deposited with this note in the program's certificate repository *Certify*, and a reader with nothing but CPython replays any shipped bracket in seconds ($W_{\max} = 5$) from the certificate alone. The generator, the enumeration engine, and the Monte Carlo remain private and are not needed to check anything.

7. Questions

Question 7.1 (Break the weight-seven wall). Is there a proof-carrying counter for the exact weighted enumeration at $W_{\max} = 7$ — a CPOG-style weighted model-counting certificate, or an exact locality-exploiting convolution / transfer-matrix — that a standard-library checker can re-verify in minutes rather than hours? This is the difference between a bracket that stops at $d = 5$ and one that scales.

Question 7.2 (Larger distance). How does the object behave at $d = 7$ and beyond? Even at one round the mechanism count and the syndrome space 2^n grow, so both the decoder rebuild and the enumeration $\binom{m}{w}$ swell; the depth-truncated BFS keeps the decoder tractable, but the enumeration is the binding constraint, which returns to Question 7.1.

Question 7.3 (Other codes, other noise, and the circuit gap). The bracket is generic in the DEM: it applies to any code, any single logical observable, any independent-mechanism model, and — with a larger syndrome space — to multiple rounds. Two sharper questions follow. Can the truncation tail be replaced by a tighter certified tail that exploits the mechanism structure? And can the DEM-to-circuit gap of §5 itself be certified, lifting the bracket from Stim's model to the physical depolarizing circuit?

Question 7.4 (The decoder gap). The bracket bounds a fixed minimum-cardinality lookup decoder, not the optimal decoder. Uncorrectable-set counting is decoder-relative by construction; is there a certified two-sided bracket on the *maximum-likelihood* decoder's logical rate — the code's true operational figure — of comparable sub-threshold sharpness?

Acknowledgments

The question this note answers — that sub-threshold logical error rates are beyond direct Monte Carlo — is Mullan, Weippert, and Brown's, and their subregion-MCMC estimator is the sampler this certificate is meant to complement, not replace; the Markov-chain lineage traces to Bravyi and Vargo. The lookup-table decoder is Tomita and Svore's. The detector error models were produced with Stim (Craig Gidney), used here as an untrusted generator. The computation and drafting were AI-assisted as stated in the first footnote, and the note's shape — open with the question, cite the computation once, close with the questions it opens — follows the program's standing constraint for a computation-only paper.

References

[BV14] S. Bravyi and A. Vargo, *Simulation of rare events in quantum error correction*, Phys. Rev. A **88** (2013), 062308; arXiv:1308.6270.

[FVC25] D. Forlivesi, L. Valentini and M. Chiani, *Performance analysis of quantum error-correcting codes via MacWilliams identities*, Quantum **9** (2025), 1950; arXiv:2305.01301.

[LeanQEC26] K. Ehatamm, S. Lee, Y. Wu and R. Tao, *Lean-QEC: machine-checked quantum error-correcting code distances via a verified SAT reduction*, arXiv:2605.16523 (2026).

[MWB26] M. Mullan, M. Weippert and W. Brown, *Improved methods for determining quantum error correcting code performance and fault tolerance*, arXiv:2607.27153 (2026).

[Reg26] G. Regev, G. Dilley, T. Nutaro, R. Delgado and R. Bennink, *Closed form logical error rate approximations for surface codes*, arXiv:2605.03054 (2026).

[TS14] Y. Tomita and K. M. Svore, *Low-distance surface codes under realistic quantum noise*, Phys. Rev. A **90** (2014), 062320.

[VeriQEC25] C. Huang et al., *Veri-QEC: verifying error correction and detection conditions for quantum error correcting codes*, Proc. ACM Program. Lang. (2025), DOI 10.1145/3729293; arXiv:2504.07732.

Independent researcher (05oz), Half Ounce Research; no institutional affiliation. ORCID: 0009-0009-5213-4098.

Email address: daniel@halfounce.io

URL: https://halfounce.io